ASE CHAPTER 3

T1 Multi Choice Questions Pack · SW Requirements

Section 1: Publisher Companion Slide MCQs (25 Items)

Q3.1. Which one is not a step of requirement engineering?

(a) Requirements elicitation · (b) Requirements analysis · (c) Requirements design · (d) Requirements documentation

> **(c) Requirements design**
>
> **Requirements engineering consists of four crucial process steps: requirements elicitation, requirements analysis, requirements documentation (producing the SRS), and requirements review (validation).** "Requirements design" is not a recognized phase; software design is a distinct subsequent phase of the life cycle that operates upon the frozen SRS.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 41; Slide 121. [Independently Checked]*

Q3.2. Requirements elicitation means:

(a) Gathering of requirements · (b) Capturing of requirements · (c) Understanding of requirements · (d) All of the above

> **(d) All of the above**
>
> **Requirements elicitation encompasses discovering, gathering, capturing, and understanding the real needs of customers, users, and organizational stakeholders.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49; Slide 121. [Independently Checked]*
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - الاستخلاص ليس مجرد "جمع" سلبي للمعلومات، بل يشمل الاستكشاف والتوضيح والاستيعاب العميق لرغبات العميل والقيود المحيطة بالنظام.

Q3.3. SRS stands for:

(a) Software requirements specification · (b) System requirements specification · (c) Systematic requirements specifications · (d) None of the above

> **(a) Software requirements specification**
>
> **SRS stands for Software Requirements Specification.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 122. [Independently Checked]*

Q3.4. SRS document is for:

(a) "What" of a system? · (b) How to design the system? · (c) Costing and scheduling of a system · (d) System's requirement.

(a) "What" of a system?

The SRS describes what the software must do, including required external behavior and interfaces, while leaving internal design and implementation decisions to the design stage.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, p. 40; Slide 122. [Independently Checked]

Q3.5. Requirements review process is carried out to:

(a) Spend time in requirements gathering · (b) Improve the quality of SRS · (c) Document the requirements · (d) None of the above

(b) Improve the quality of SRS

The requirements review process is conducted to detect and correct omissions, ambiguities, contradictions, and unrealizable requirements, thereby directly improving the technical quality and baseline integrity of the SRS.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1.1, p. 41; Slide 123. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- مراجعة المتطلبات (Requirements Review) هي نشاط توكيد جودة يهدف لكشف العيوب في الـ SRS قبل الانتقال للتصميم لتفادي كلفة الإصلاح المضاعفة لاحقاً.

Q3.6. Which one of the statements is not correct during requirements engineering?

(a) Requirements are difficult to uncover · (b) Requirements are subject to change · (c) Requirements should be consistent · (d) Requirements are always precisely known.

(d) Requirements are always precisely known.

In empirical software engineering practice, requirements are rarely precisely known upfront; customers often have vague initial concepts, requirements constantly change, and discovering complete needs requires disciplined iteration.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.1, pp. 40–41; Slide 123. [Independently Checked]

Q3.7. Which one is not a type of requirements?

(a) Known requirements · (b) Unknown requirements · (c) Undreamt requirements · (d) Complex requirements

(d) Complex requirements

The textbook categorizes requirements discovery into three states: known requirements (explicitly understood and stated), unknown requirements (revealed during investigation), and undreamt requirements (innovative features beyond user expectations). "Complex requirements" is not a recognized classification category in this model.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2, p. 45; Slide 124. [Independently Checked]

Q3.8. Which one is not a requirements elicitation technique?

(a) Interviews · (b) The use case approach · (c) FAST · (d) Data flow diagram.

(d) Data flow diagram.

Interviews, the use case approach, and FAST are requirements elicitation techniques used to capture user needs. A Data Flow Diagram (DFD) is a structural requirements analysis and data-modeling tool, not an elicitation technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4, p. 49 & §3.5.1, p. 60; Slide 124. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- ميز بين أدوات الاستخلاص (Interviews, Use Cases, FAST, QFD) وأدوات التحليل والنمذجة الهيكلية (DFD, ERD, Data Dictionary).

Q3.9. FAST stands for:

(a) Functional Application Specification Technique · (b) Fast Application Specification Technique · (c) Facilitated Application Specification Technique · (d) None of the above

(c) Facilitated Application Specification Technique

FAST stands for Facilitated Application Specification Technique.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.3, p. 51; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- كلمة FAST تعني Facilitated (ميسّرة)، وهي جلسات عمل مشتركة ومكثفة بين العملاء والمطورين يقودها ميسر محايد.

Q3.10. QFD in requirement engineering stands for:

(a) Quality function design · (b) Quality factor design · (c) Quality function development · (d) Quality function deployment

(d) Quality function deployment

QFD stands for Quality Function Deployment.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 125. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- احفظ المصطلح بدقة: Deployment (نشر/تطبيق وظيفة الجودة)، وترجمتها لمتطلبات هندسية ملموسة.

Q3.11. Which is not a type of requirements under quality function deployment:

(a) Normal requirements · (b) Abnormal requirements · (c) Expected requirements · (d) Exciting requirements

(b) Abnormal requirements

Under Quality Function Deployment (QFD), requirements are structured into Normal requirements (stated objectives), Expected requirements (implicit, baseline requirements), and Exciting requirements (innovative features that delight the customer). "Abnormal requirements" is not a QFD category.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.4, p. 53; Slide 126. [Independently Checked]

Q3.12. Use case approach was developed by:

(a) I. Jacobson and others · (b) J.D. Musa and others · (c) B. Littlewood · (d) None of the above

> **(a) I. Jacobson and others**
>
> **The Use Case Approach was developed by Ivar Jacobson and his colleagues (1992) as a scenario-driven method for capturing system behavior from the viewpoint of external actors.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slide 126. [Independently Checked]*

Q3.13. Context diagram explains:

(a) The overview of the system · (b) The internal view of the system · (c) The entities of the system · (d) None of the above

> **(a) The overview of the system**
>
> **A context diagram represents the highest-level structural overview of the system, abstracting the entire system as a single process bubble interacting with external environment entities.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5, p. 59; Slide 127. [Independently Checked]*

Q3.14. DFD stands for:

(a) Data Flow design · (b) Descriptive functional design · (c) Data flow diagram · (d) None of the above

> **(c) Data flow diagram**
>
> **DFD stands for Data Flow Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 127. [Independently Checked]*

Q3.15. Level-0 DFD is similar to:

(a) Use case diagram · (b) Context diagram · (c) System diagram · (d) None of the above

> **(b) Context diagram**
>
> **In structured analysis, a Level-0 DFD is synonymous with the Context Diagram, modeling the whole target application as single process 0.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 61; Slide 128. [Independently Checked]*

Q3.16. ERD stands for:

(a) Entity relationship diagram · (b) Exit related diagram · (c) Entity relationship design · (d) Exit related design

> **(a) Entity relationship diagram**
>
> **ERD stands for Entity Relationship Diagram.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 128. [Independently Checked]*

Q3.17. Which is not a characteristic of a good SRS?

(a) Correct · (b) Complete · (c) Consistent · (d) Brief

(d) Brief

IEEE Standard 830-1998 specifies that a good SRS must be correct, unambiguous, complete, consistent, ranked for importance/stability, verifiable, modifiable, and traceable. Brevity is not a standard quality criterion; striving for shortness frequently introduces omissions and ambiguities.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, p. 73; Slide 129. [Independently Checked]

Q3.18. Outcome of requirements specification phase is:

(a) Design Document · (b) Software requirements specification · (c) Test Document · (d) None of the above

(b) Software requirements specification

The deliverable produced by the requirements specification phase is the Software Requirements Specification (SRS).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6, p. 72; Slide 129. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- وثيقة SRS هي الناتج الرسمي لمرحلة المواصفات، وهي العقد الفاصل بين متطلبات المستخدم وبداية التصميم المعماري.

Q3.19. The basic concepts of ER model are:

(a) Entity and relationship · (b) Relationships and keys · (c) Entity, effects and relationship · (d) Entity, relationship and attribute

(d) Entity, relationship and attribute

The three foundational modeling concepts of the Entity-Relationship (ER) model are Entities (data objects), Relationships (associations among entities), and Attributes (properties qualifying entities or relationships).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.3, p. 63; Slide 130. [Independently Checked]

Q3.20. The DFD depicts:

(a) Flow of data · (b) Flow of control · (c) Both (a) and (b) · (d) None of the above

(a) Flow of data

A Data Flow Diagram depicts solely the flow and transformation of data through system processes; it contains no control flow, decision sequence, or loop logic.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, p. 60; Slide 130. [Independently Checked]

Q3.21. Product features are related to:

(a) Functional requirements · (b) Non functional requirements · (c) Interface requirement · (d) None of the above

(a) Functional requirements

Product features describe the operational capabilities, services, and functional transformations provided by the software to its end users, defining functional requirements.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.22. Which one is a quality attribute?

(a) Reliability · (b) Availability · (c) Security · (d) All of the above

(d) All of the above

Reliability, availability, and security are examples of non-functional quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 131. [Independently Checked]

Q3.23. IEEE standard for SRS is:

(a) IEEE Standard 837-1998 · (b) IEEE Standard 830-1998 · (c) IEEE Standard 832-1998 · (d) IEEE Standard 839-1998

(b) IEEE Standard 830-1998

The canonical standard for Software Requirements Specifications established in course literature is IEEE Standard 830-1998 (Recommended Practice for Software Requirements Specifications).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.2, p. 46; Slide 132. [Independently Checked]

Q3.24. Which one is not a functional requirement?

(a) Efficiency · (b) Reliability · (c) Product features · (d) Stability

No unique answer: (a) Efficiency, (b) Reliability, and (d) Stability are all non-functional requirements.

Product features represent functional requirements, whereas efficiency, reliability, and stability are non-functional quality attributes. In this publisher slide item (Slide 132), the question contains a recognized structural flaw in its negative phrasing: three options ((a) Efficiency, (b) Reliability, (d) Stability) are non-functional requirements, while only (c) Product features is functional. If the examiner asks "*Which one is a functional requirement?*", the intended answer is (c). If asked verbatim as printed, any of (a), (b), or (d) is technically correct, with Reliability and Efficiency being canonical textbook quality attributes.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.1, pp. 45–46; Slide 132. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- **تنبيه لعيب صياغة المصدر:** هذا السؤال ورد نصاً بالسلايد 132 بصياغة ملتبسة؛ فخيارات (Efficiency, Reliability, Stability) كلها متطلبات غير وظيفية (Non-Functional / Quality Attributes)، بينما (Product features) هو الوحيد الوظيفي. إذا جاء السؤال كما هو، فإن (a) أو (b) أو (d) صحيحة بالمعنى الحرفي، أما إذا عُدلت الصياغة في الامتحان إلى "*Which one IS a functional requirement?*" فالجواب هو قطعاً (c).

MCQ-25 (Slide 133, printed as duplicate 3.23). APIs stand for:

(a) Application performance interfaces · (b) Application programming interfaces · (c) Application programming integration · (d) Application performance integration

(b) Application programming interfaces

APIs stand for Application Programming Interfaces, which establish procedural interface specifications for programmatic inter-software communication.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 133. [Independently Checked]

ملاحظة الطالب والفخ الامتحاني:

- API = Application Programming Interface، وهو يمثل نمطاً للمواصفات الإجرائية (Procedural Interface).
  الرقم طُبع مكرراً كـ 3.23 بالسلايد 133 وثبتناه كـ MCQ-25.

Section 2: Additional Chapter 3 Synthesis MCQs (10 Items)

Q3.26. What are the three primary types of interface specifications required in software engineering?

(a) User interface, Database interface, and Network interface · (b) Procedural interfaces (APIs), Data structures, and Representation of data · (c) Hardware interface, Firmware interface, and Software interface · (d) Input interface, Output interface, and Storage interface

(b) Procedural interfaces (APIs), Data structures, and Representation of data

The three types of interface specifications are procedural interfaces, data structures, and representation of data.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.2.3, p. 46; Slide 9.

ملاحظة الطالب والفخ الامتحاني:

- التزم حرفياً بتبويب الشريحة 9 المعتمدة للدكتور: الأنواع الثلاثة لمواصفات الواجهات هي: الواجهات الإجرائية (Procedural interfaces - APIs)، هياكل البيانات (Data structures)، وتمثيل البيانات (Representation of data).
  لا تخلط بين هذا التبويب الخاص بالسلايد وبين تبويب معيار IEEE 830 الخارجي (User, Hardware, Software, Communications).

Q3.27. Under a Feasibility Study, which dimension evaluates whether the development organization has the necessary tools, platforms, and technical expertise?

(a) Operational feasibility · (b) Economic feasibility · (c) Technical feasibility · (d) Schedule feasibility

(c) Technical feasibility

Technical feasibility determines whether the proposed application can be constructed using available technology, whether technical constraints can be satisfied, and whether the development team possesses the requisite technical competencies.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.3.1, pp. 47–48; Slides 11–13.

Q3.28. What is the bounded objective of risk identification within the Feasibility Study phase according to Chapter 3 lecture materials?

(a) Executing full spiral risk cycles · (b) Determining whether major project risks are identified and can be successfully surmounted · (c) Eliminating all project uncertainties permanently · (d) Purchasing commercial insurance policies

(b) Determining whether major project risks are identified and can be successfully surmounted

Within the feasibility study, the focus is on determining whether major project risks have been identified and whether they can be surmounted before the project proceeds.

Source: Chapter 3 lecture, Slide 14; Dr. Ali Fahim Vocabulary Ledger (2026-09-30).

ملاحظة الطالب والفخ الامتحاني:

- انتبه لحدود إدارة المخاطر في هذا الفصل: ليست دورة حلزونية كاملة، بل التحقق من أن المخاطر الجوهرية محددة وقابلة للتجاوز (Surmountable) كشرط للمضي بالمشروع.

Q3.29. How do User Requirements fundamentally differ from System Requirements?

(a) User requirements are written in code; system requirements are written in English · (b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers · (c) User requirements describe non-functional constraints only; system requirements describe functions only · (d) User requirements are authored during testing; system requirements during elicitation

(b) User requirements state high-level customer expectations in natural language and diagrams, while system requirements define structured, detailed functional and operational specifications for engineers

User requirements are high-level statements in natural language and informal diagrams defining what services the system provides and its operational constraints for clients and management. System requirements are comprehensive, structured descriptions of software functions, services, and operational constraints serving as the contract and basis for architectural design.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Slide 8; Sommerville (9th ed.), §4.1, pp. 84–85.

Q3.30. In Data Flow Diagramming (DFD), what constitutes a "Black Hole" defect?

(a) A process that produces outputs without receiving any inputs · (b) A process that has input data flows but generates no output data flows · (c) A data store that is never accessed · (d) A data flow connecting two external entities

(b) A process that has input data flows but generates no output data flows

In DFD rule verification, a "Black Hole" refers to a process that consumes input data flows but produces zero output data flows, violating the fundamental principle of data transformation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; B.B. Agarwal et al. (2010), §3.3.1, p. 57.

ملاحظة الطالب والفخ الامتحاني:

- الثقب الأسود (Black Hole) في الـ DFD: عملية تدخلها بيانات دون أن يخرج منها أي تدفق. عكسها المعجزة (Miracle): عملية تنتج مخرجات دون أي مدخلات!

Q3.31. Which data flow connection is strictly illegal in a valid Data Flow Diagram?

(a) Process to Process · (b) Process to Data Store · (c) Data Store directly to another Data Store without an intervening Process · (d) External Entity to Process

(c) Data Store directly to another Data Store without an intervening Process

Data cannot move autonomously between data stores or between external entities; every valid data flow in a DFD must have a Process as either its origin or its destination.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.1, pp. 60–61; Slides 85–86.

Q3.32. In the course Data Dictionary notation, what does { } indicate?

(a) An optional data element · (b) Selection of one alternative from a list · (c) Repetition of a data element, with adjacent bounds specifying its permitted count · (d) Sequential composition of data elements

(c) Repetition of a data element, with adjacent bounds specifying its permitted count

In Table 3.3, x=a+b means x consists of a and b; x=[a/b] means either a or b; and x=(a) marks an optional item. Braces denote repetition: x=y{a}z means between y and z occurrences of a, while x=y{a} and x={a}z express lower-only and upper-only bounds, respectively.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.5.2, Table 3.3, p. 63; Slide 88; B.B. Agarwal et al. (2010), §3.3.2, p. 58.

ملاحظة الطالب والفخ الامتحاني:

- تنبيه مصدري: جدول 3.3 في كتاب أغروال يكتب الاختيار بين البدائل هكذا x=[a/b]، بينما سلايد 88 يستخدم {a/b}؛ اعتمدنا صيغة الكتاب لأنه المصدر الأساسي. أما التكرار المحدود في الكتاب فيظهر بصيغة x=y{a}z.

Q3.33. A requirement states: "*The software shall be intuitive, fast, and extremely user-friendly.*" Which IEEE 830 characteristic is directly violated?

(a) Modifiability · (b) Verifiability · (c) Traceability · (d) Ranking for importance

(b) Verifiability

A requirement is verifiable if and only if there exists a finite, cost-effective process by which a person or machine can measure whether the delivered software meets the requirement. Subjective adjectives like "intuitive", "fast", and "user-friendly" cannot be objectively verified or tested without quantifiable metrics (such as task completion time or maximum click count).

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.6.2, pp. 74–75; Slide 104.

Q3.34. In Use Case modeling, what is the defining distinction between an <<include>> and an <<extend>> relationship?

(a) <<include>> is optional; <<extend>> is mandatory · (b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points · (c) <<include>> connects actors only; <<extend>> connects use cases only · (d) They are functionally identical

(b) <<include>> unconditionally executes shared common behavior, while <<extend>> conditionally adds optional behavior at defined extension points

An <<include>> relationship represents mandatory factoring of common subroutines that execute unconditionally during the base use case. An <<extend>> relationship represents optional or exceptional behavior that is executed only if specific trigger conditions are met at declared extension points.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §3.4.5, p. 54; Slides 30–33.

Q3.35. What is the fundamental difference between Requirements Verification and Requirements Validation?

(a) Verification checks code syntax; Validation checks compiler optimization · (b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs) · (c) Verification is performed exclusively by end users; Validation is performed by software developers · (d) Verification is dynamic testing; Validation is static review

(b) Verification asks "Are we building the product right?" (conformance to specification), while Validation asks "Are we building the right product?" (satisfying actual customer needs)

Verification ensures that intermediate engineering artifacts strictly adhere to preceding lifecycle specifications (conformance). Validation ensures that the specified system accurately satisfies the real-world operational intentions and operational objectives of the customer.

Source: Slide 111; B.B. Agarwal et al. (2010), §3.8, p. 68.